

LAPORAN PROJECT UJIAN AKHIR SEMESTER

MATA KULIAH DATA WAREHOUSE

**PERANCANGAN DAN IMPLEMENTASI DATA WAREHOUSE
PENGELOLAAN DATA PENJUALAN *BRAZILIAN E-COMMERCE BY OLIST*
MENGUNAKAN STAR SCHEMA DAN PROSES ETL**

Dosen Pengampu:

Endah Septa Sintiya, S.Pd., M.Kom.



Disusun Oleh:

Dimas Adit Thalia Putra	244107060037
Fiza Rahmatus Sholikha	244107060109
Diyah Ramadhani Putri	244107060152
Siti Aisyah	244107060038
Kevin Marsha H.A	244107060077

KELAS 2E

PROGRAM STUDI D-IV SISTEM INFORMASI DAN BISNIS

JURUSAN TEKNOLOGI INFORMASI

POLITEKNIK NEGERI MALANG

TAHUN 2026

BAB I

STUDI KASUS DAN SUMBER DATA

1.1 Deskripsi Studi Kasus

Olist adalah startup e-commerce asal Brasil yang beroperasi sebagai marketplace, menghubungkan ribuan penjual kecil (merchant) dengan jutaan pelanggan di seluruh negara. Platform ini memungkinkan para penjual untuk mendaftarkan produk mereka dan melayani pembeli melalui satu kanal terintegrasi.

Dataset Brazilian E-Commerce Public Dataset by Olist merupakan dataset publik yang dirilis oleh Olist di platform Kaggle. Dataset ini berisi informasi transaksi nyata dari sekitar 100.000 pesanan yang dilakukan antara tahun 2016 hingga 2018. Data mencakup berbagai aspek transaksi, mulai dari informasi pelanggan, produk yang dijual, penjual, status pesanan, hingga data logistik dan ulasan pembeli.

Dataset ini sangat cocok digunakan sebagai studi kasus data warehouse karena memiliki struktur relasional yang kompleks, berasal dari sistem OLTP (Online Transaction Processing) yang nyata, dan mencerminkan karakteristik data e-commerce yang umum dijumpai di industri. Dengan mengintegrasikan dataset ini ke dalam data warehouse, kita dapat menganalisis berbagai KPI bisnis yang relevan seperti tren penjualan, performa kategori produk, dan distribusi geografis pelanggan.

1.2 Sumber Dataset

Dataset Olist dapat diakses secara gratis di Kaggle melalui tautan berikut: <https://www.kaggle.com/datasets/olistbr/brazilian-ecommerce>.

Dataset ini berlisensi CC BY-NC-SA 4.0 dan terdiri dari beberapa file CSV yang saling berelasi sebagai berikut:

- olist_customers_dataset.csv berisi data demografi pelanggan.
- olist_sellers_dataset.csv berisi data identitas dan lokasi penjual.
- olist_products_dataset.csv berisi data atribut produk termasuk kategori dalam bahasa Portugis.
- product_category_name_translation.csv berisi terjemahan nama kategori dari bahasa Portugis ke bahasa Inggris.
- olist_orders_dataset.csv berisi data pesanan termasuk timestamp pembelian.
- olist_order_items_dataset.csv berisi detail item per pesanan termasuk harga dan ongkos kirim.

Seluruh file tersebut kemudian dimasukkan ke dalam database OLTP MySQL bernama db_oltp_olist. Database ini berjalan pada server lokal dengan alamat localhost dan port 3306. Dalam proyek ini, database db_oltp_olist digunakan sebagai sumber data utama pada proses ekstraksi ETL sebelum data diolah dan dimuat ke dalam data warehouse.

1.3 Deskripsi Tabel Sumber

Berikut ini adalah tabel ringkasan deskripsi masing-masing tabel yang digunakan beserta fungsi data dan atribut pentingnya:

Nama Tabel	Fungsi Data	Atribut Penting
olist_customers_dataset	Data identitas dan lokasi pelanggan	- customer_id - customer_zip_code_prefix - customer_city - customer_state
olist_sellers_dataset	Data identitas dan lokasi penjual	- seller_id - seller_zip_code_prefix - seller_city, seller_state
olist_products_dataset	Data atribut dan kategori produk	- product_id - product_category_name - product_weight_g - product_length_cm
product_category_name_translation	Terjemahan nama kategori ke bahasa Inggris	- product_category_name - product_category_name_english
olist_orders_dataset	Data header pesanan dan timestamp transaksi	- order_id, customer_id - order_status - order_purchase_timestamp
olist_order_items_dataset	Detail item per pesanan (harga dan ongkos kirim)	- order_id - product_id - seller_id - price - freight_value

Hubungan antar tabel sumber mengikuti pola relasional database OLTP, di mana order_id menjadi penghubung antara tabel orders dan order_items, serta customer_id, product_id, dan seller_id masing-masing menghubungkan ke tabel dimensi yang sesuai.

BAB II

PERANCANGAN DATA WAREHOUSE

2.1 Konsep Star Schema

Star Schema adalah pendekatan desain skema database yang paling umum digunakan dalam data warehouse. Disebut star schema (skema bintang) karena bentuk visualisasinya menyerupai bintang, di mana sebuah tabel fakta (fact table) berada di tengah dan dikelilingi oleh beberapa tabel dimensi (dimension tables) yang terhubung melalui foreign key.

Karakteristik utama star schema adalah sebagai berikut:

- **Tabel Fakta (Fact Table):** Tabel yang berada di pusat skema, berisi metrik kuantitatif (seperti harga dan ongkos kirim) serta foreign key yang mengacu ke seluruh tabel dimensi. Tabel fakta menyimpan data transaksi dalam jumlah besar.
- **Tabel Dimensi (Dimension Table):** Tabel yang berada di sekeliling tabel fakta, berisi data deskriptif yang memberikan konteks terhadap data fakta (seperti data pelanggan, produk, waktu, dan penjual). Tabel dimensi biasanya lebih kecil namun lebih lebar (banyak atribut).

Keunggulan star schema dibandingkan model lain seperti snowflake schema antara lain adalah query yang lebih sederhana dan mudah dipahami, performa query yang lebih cepat karena minimnya jumlah join, serta kemudahan dalam pembuatan laporan dan dashboard analitik.

2.2 Rancangan Tabel Dimensi

1. Dim_Pelanggan

Tabel Dim_Pelanggan digunakan untuk menyimpan informasi demografis dan lokasi pelanggan. Tabel ini memungkinkan analisis distribusi penjualan berdasarkan wilayah geografis pelanggan, seperti kota dan provinsi. Data ini diambil dari tabel `olist_customers_dataset`. Berikut adalah atribut-atribut pada `dim_pelanggan`:

- `id_pelanggan` (Primary Key): Berasal dari `customer_id`.
- `kode_pos_pelanggan`: Berasal dari data nomor kodepos `customer_zip_code_prefix`.
- `kota_pelanggan`: Berasal dari data nama kota `customer_city`.
- `provinsi_pelanggan`: Berasal dari data kode wilayah `customer_state`.

2. Dim_Penjual

Tabel Dim_Penjual menyimpan informasi identitas dan lokasi asal setiap penjual yang terdaftar di platform Olist. Tabel ini berguna untuk menganalisis distribusi geografis penjual serta performa penjualan berdasarkan wilayah asal penjual. Data ini diambil dari tabel olist_sellers_dataset. Berikut adalah atribut-atribut pada dim_penjual:

- id_penjual (Primary Key): Berasal dari seller_id.
- kode_pos_penjual: Berasal dari data nomor kodepos seller_zip_code_prefix.
- kota_penjual: Berasal dari data nama kota seller_city.
- provinsi_penjual: Berasal dari data kode wilayah seller_state.

3. Dim_Produk

Tabel Dim_Produk menyimpan informasi atribut setiap produk yang pernah terjual, termasuk kategori produk yang sudah diterjemahkan ke dalam bahasa Inggris dari hasil join dengan tabel translasi, serta dimensi fisik produk. Tabel ini memungkinkan analisis penjualan berdasarkan kategori dan karakteristik produk. Data ini diambil dari tabel olist_products_dataset dengan join tabel product_category_name_translation. Berikut adalah atribut-atribut pada dim_produk:

- id_produk (Primary Key): Berasal dari product_id.
- kategori_produk: Berasal dari konversi data product_category_name menggunakan kolom terjemahan product_category_name_english.
- berat_produk_gram: Berasal dari data ukuran berat fisik product_weight_g.
- panjang_produk_cm: Berasal dari data dimensi ukuran panjang product_length_cm.

4. Dim_Waktu

Tabel Dim_Waktu adalah tabel dimensi khusus yang berisi informasi detail tentang waktu transaksi. Data ini diekstraksi dari kolom order_purchase_timestamp pada tabel pesanan. Kehadiran tabel waktu yang terpisah memungkinkan query berbasis periode harian, bulanan, kuartalan, tahunan menjadi jauh lebih cepat dan efisien. Berikut adalah atribut-atribut pada dim_waktu:

- id_waktu (Primary Key)
- tanggal_lengkap: Berasal dari ekstraksi tipe data tanggal penuh order_purchase_timestamp.
- hari: Angka urutan hari (1-31) yang diekstrak dari kolom waktu order_purchase_timestamp.

- bulan: Angka urutan bulan (1-12) yang diekstrak dari kolom waktu order_purchase_timestamp.
- tahun: Angka digit tahun transaksi yang diekstrak dari kolom waktu order_purchase_timestamp.
- kuartal: Pengelompokan masa triwulan dalam setahun (Q1, Q2, Q3, Q4) berdasarkan nilai bulan transaksinya.

2.3 Rancangan Tabel Fakta

1. Fact_Penjualan

Fact_Penjualan adalah tabel inti yang berada di pusat star schema. Tabel ini menyimpan seluruh data transaksi penjualan beserta metrik numerik yang dapat dianalisis. Setiap baris pada tabel ini merepresentasikan satu item produk yang terjual dalam satu pesanan tertentu. Data ini diambil dari tabel olist_orders_dataset dengan join tabel olist_order_items_dataset. Berikut adalah atribut-atribut pada Fact_Penjualan:

- id_fakta (Primary Key): Nomor urut unik otomatis (Auto Increment) khusus untuk data warehouse.
- id_pesanan: Berasal dari nomor identitas transaksi order_id.
- id_pelanggan (Foreign Key): Menghubungkan transaksi ke data primer di tabel Dim_Pelanggan.
- id_produk (Foreign Key): Menghubungkan transaksi ke data primer di tabel Dim_Produk.
- id_penjual (Foreign Key): Menghubungkan transaksi ke data primer di tabel Dim_Penjual.
- id_waktu (Foreign Key): Menghubungkan transaksi ke data primer di tabel Dim_Waktu.
- harga_barang: Metrik nilai nominal harga komoditas utama dari kolom data price.
- ongkos_kirim: Metrik nilai nominal beban biaya distribusi logistik dari kolom data freight_value.

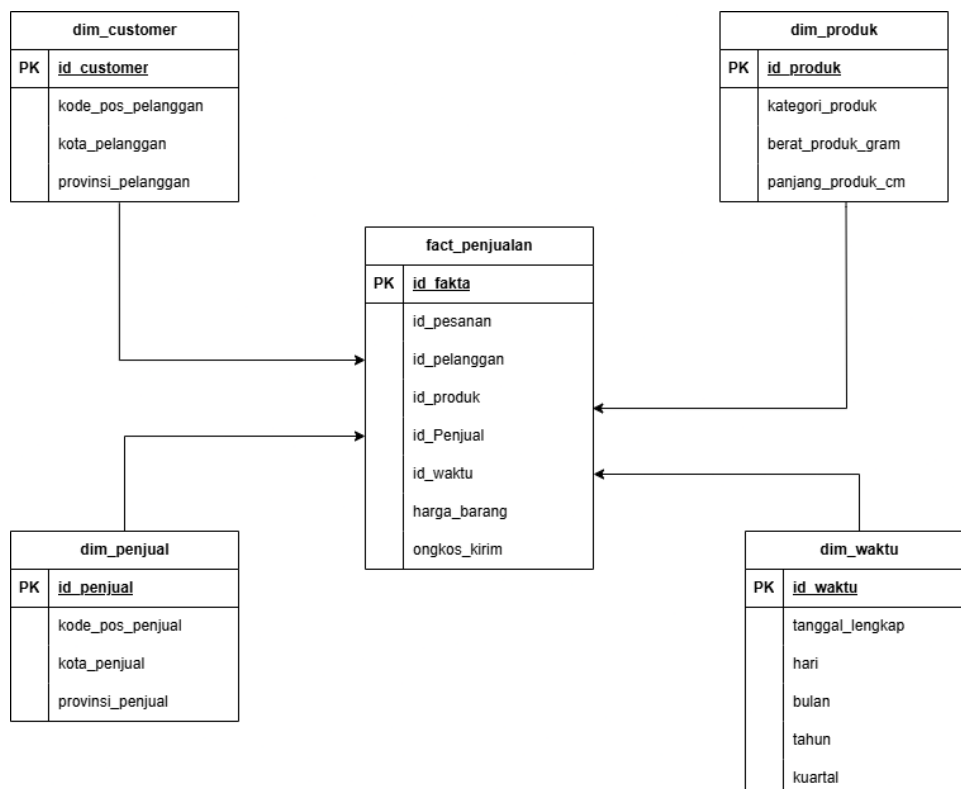
2.4 Relasi Antar Tabel

Berikut ini adalah penjelasan relasi antara Fact_Penjualan dengan setiap tabel dimensi dalam star schema yang telah dirancang:

Tabel Fakta	Tabel Dimensi	Hubungan (Foreign Key)
Fakta_Penjualan	Dim_Pelanggan	Fact_Penjualan.id_pelanggan dengan Dim_Pelanggan.id_pelanggan
Fakta_Penjualan	Dim_Produk	Fact_Penjualan.id_produk dengan Dim_Produk.id_produk
Fakta_Penjualan	Dim_Penjual	Fact_Penjualan.id_penjual dengan Dim_Penjual.id_penjual
Fakta_Penjualan	Dim_Waktu	Fact_Penjualan.id_waktu dengan Dim_Waktu.id_waktu

Setiap foreign key pada tabel Fact_Penjualan bersifat NOT NULL, artinya setiap transaksi harus memiliki referensi yang valid ke semua tabel dimensi. Hal ini menjamin integritas data dalam data warehouse dan memastikan tidak ada transaksi yang kehilangan konteks dimensinya.

2.5 Gambar Star Schema



BAB III

IMPLEMENTASI ETL

3.1 Pembuatan Database

1. Create Database

```
1 CREATE DATABASE dw_penjualan_olist
```

2. Create Tabel dim_produk

```
1 CREATE TABLE Dim_Produk (  
2     id_produk VARCHAR(50) PRIMARY KEY,  
3     kategori_produk VARCHAR(100),  
4     berat_produk_gram INT,  
5     panjang_produk_cm INT  
6 );
```

3. Create Tabel dim_pelanggan

```
1 CREATE TABLE Dim_Pelanggan (  
2     id_pelanggan VARCHAR(50) PRIMARY KEY,  
3     kode_pos_pelanggan VARCHAR(20),  
4     kota_pelanggan VARCHAR(100),  
5     provinsi_pelanggan VARCHAR(5)  
6 );
```

4. Create Tabel dim_penjual

```
1 CREATE TABLE Dim_Penjual (  
2     id_penjual VARCHAR(50) PRIMARY KEY,  
3     kode_pos_penjual VARCHAR(20),  
4     kota_penjual VARCHAR(100),  
5     provinsi_penjual VARCHAR(5)  
6 );
```

5. Create Tabel dim_waktu

```
CREATE TABLE Dim_Waktu (  
    id_waktu INT PRIMARY KEY AUTO_INCREMENT,  
    tanggal_lengkap DATE,  
    hari INT,  
    bulan INT,  
    tahun INT,  
    kuartal VARCHAR(2)  
)
```

6. Create Tabel fact_penjualan

```
1 CREATE TABLE Fact_Penjualan (  
2     id_fakta INT AUTO_INCREMENT PRIMARY KEY,  
3     id_pesanan VARCHAR(50),  
4     id_pelanggan VARCHAR(50),  
5     id_produk VARCHAR(50),  
6     id_penjual VARCHAR(50),  
7     id_waktu INT,  
8     harga_barang DECIMAL(10, 2),  
9     ongkos_kirim DECIMAL(10, 2),  
10    FOREIGN KEY (id_pelanggan) REFERENCES Dim_Pelanggan(id_pelanggan),  
11    FOREIGN KEY (id_produk) REFERENCES Dim_Produk(id_produk),  
12    FOREIGN KEY (id_penjual) REFERENCES Dim_Penjual(id_penjual),  
13    FOREIGN KEY (id_waktu) REFERENCES Dim_Waktu(id_waktu)  
14 );
```


3.2 Konfigurasi Database Connection

1. Database oltp

The screenshot shows the 'Database Connection' dialog box with the 'General' tab selected. The 'Connection name' is 'conn_oltp_olist'. The 'Connection type' is 'MySQL'. The 'Access' is 'Native (JDBC)'. The 'Settings' section includes: 'Host Name' (localhost), 'Database Name' (db_oltp_olist), 'Port Number' (3306), 'Username' (root), and 'Password' (empty). The 'Use Result Streaming Cursor' checkbox is checked. The 'Test', 'Feature List', and 'Explore' buttons are visible at the bottom.

Database Connection

General
Advanced
Options
Pooling
Clustering

Connection name:
conn_oltp_olist

Connection type:
MySQL
Native Mondrian
Neoview (deprecated)
Netezza
Oracle
Oracle RDB
PostgreSQL
Redshift
Remedy Action Request System
SAP ERP System
SQLite
Snowflake

Access:
Native (JDBC)
JNDI

Settings
Host Name:
localhost
Database Name:
db_oltp_olist
Port Number:
3306
Username:
root
Password:

☒ Use Result Streaming Cursor

Test Feature List Explore

OK Cancel

2. Database olap

The screenshot shows the 'Database Connection' dialog box with the 'General' tab selected. The 'Connection name' is 'conn_olap_olist'. The 'Connection type' is 'MySQL'. The 'Access' is 'Native (JDBC)'. The 'Settings' section includes: 'Host Name' (localhost), 'Database Name' (dw_penjualan_olist), 'Port Number' (3306), 'Username' (root), and 'Password' (empty). The 'Use Result Streaming Cursor' checkbox is checked. The 'Test', 'Feature List', and 'Explore' buttons are visible at the bottom.

Database Connection

General
Advanced
Options
Pooling
Clustering

Connection name:
conn_olap_olist

Connection type:
Intersystems Cache
KingbaseES
LucidDB
MS Access
MS SQL Server
MS SQL Server (Native)
MariaDB
MaxDB (SAP DB)
MonetDB
MySQL
Native Mondrian
Neoview
Netezza
Oracle
Oracle RDB
Palo MOLAP Server
PostgreSQL

Access:
Native (JDBC)
ODBC
JNDI

Settings
Host Name:
localhost
Database Name:
dw_penjualan_olist
Port Number:
3306
Username:
root
Password:

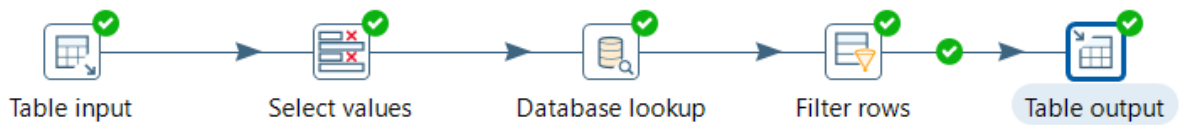
☒ Use Result Streaming Cursor

Test Feature List Explore

OK Cancel

3.3 Implementasi ETL Pada Tabel Dimensi

1. Tabel Dim_Produk



➤ Extract

Proses Extract dilakukan menggunakan komponen Tabel Input dengan data yang diambil langsung dari tabel-tabel di database OLTP db_oltp_olist melalui koneksi bernama conn_oltp_olist. Berikut adalah konfigurasi Tabel Input pada Dim_Produk:

Step name: Table input

Connection: conn_oltp_olist

SQL

```

SELECT
  p.product_id AS id_produk,
  COALESCE(t.product_category_name_english, 'unknown') AS kategori_produk,
  COALESCE(p.product_weight_g, 0) AS berat_produk_gram,
  COALESCE(p.product_length_cm, 0) AS panjang_produk_cm
FROM
  olist_products_dataset p
LEFT JOIN
  product_category_name_translation t
  ON p.product_category_name = t.product_category_name;
  
```

Line 10 Column 57

Store column info in step meta data ☐

Enable lazy conversion ☐

Replace variables in script? ☐

Insert data from step

Execute for each row? ☐

Limit size: 0

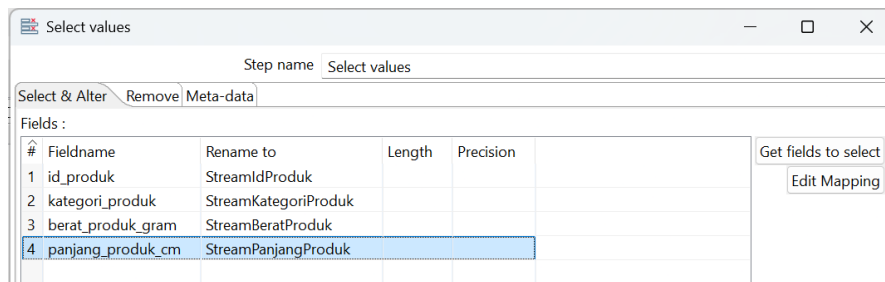
Buttons: Help, OK, Preview, Cancel

Query ini dipakai untuk step table input, Query ini mengambil pada tabel utama olist_products_dataset yang digabungkan menggunakan perintah LEFT JOIN dengan tabel product_category_name_translation untuk memastikan seluruh data produk tetap terbawa masuk meskipun tidak memiliki terjemahan kategori bahasa Inggris. Selain itu, query ini melakukan pembersihan data (data cleansing) menggunakan fungsi COALESCE di mana data kategori yang kosong (NULL) akan otomatis diubah menjadi teks 'unknown', sementara data numerik seperti berat dan panjang produk yang kosong akan diganti dengan angka 0.

➤ Transform

Pada tahap transform, data yang telah diambil dari database OLTP diolah agar sesuai dengan struktur tabel tujuan pada data warehouse

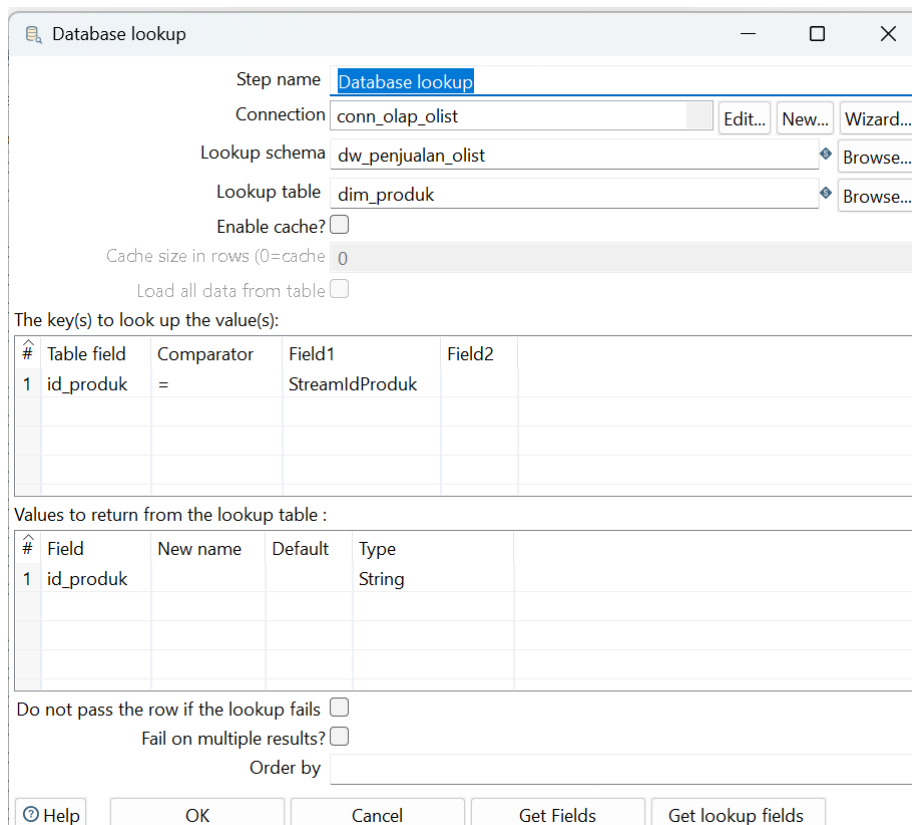
a. Konfigurasi Select Values



#	Fieldname	Rename to	Length	Precision
1	id_produk	StreamIdProduk		
2	kategori_produk	StreamKategoriProduk		
3	berat_produk_gram	StreamBeratProduk		
4	panjang_produk_cm	StreamPanjangProduk		

Komponen Select Values digunakan untuk memilih field yang diperlukan serta menyesuaikan nama kolom dengan tabel tujuan.

b. Konfigurasi database lookup



Step name: Database lookup

Connection: conn_olap_olist

Lookup schema: dw_penjualan_olist

Lookup table: dim_produk

Enable cache? ☐

Cache size in rows (0=cache): 0

Load all data from table ☐

The key(s) to look up the value(s):

#	Table field	Comparator	Field1	Field2
1	id_produk	=	StreamIdProduk	

Values to return from the lookup table :

#	Field	New name	Default	Type
1	id_produk			String

Do not pass the row if the lookup fails ☐

Fail on multiple results? ☐

Order by:

Buttons: Help, OK, Cancel, Get Fields, Get lookup fields

Database Lookup digunakan untuk mengecek apakah data sudah tersedia di tabel Dim_Product, sehingga dapat mencegah terjadinya duplikasi data.

c. konfigurasi filter rows

Step name: Filter rows

Send 'true' data to step: Table output

Send 'false' data to step:

The condition:

id_produk IS NULL

Filter Rows digunakan untuk menyaring data yang belum ada pada tabel Dim_Produk, sehingga hanya data baru yang akan diteruskan ke proses load.

➤ **Load**

Pada tahap load, data yang telah melalui proses transformasi dimasukkan ke dalam tabel tujuan di database OLAP. Proses ini menggunakan komponen Table Output dengan koneksi menuju database dw_penjualan_olist.

Step name: Table output

Connection: conn_olap_olist

Target schema: dw_penjualan_olist

Target table: dim_produk

Commit size: 1000

Truncate table: ☐

Ignore insert errors: ☐

Specify database fields: ☒

Main options Database fields

Fields to insert:

#	Table field	Stream field
1	id_produk	StreamIdProduk
2	kategori_produk	StreamKategoriProduk
3	berat_produk_gram	StreamBeratProduk
4	panjang_produk_cm	StreamPanjangProduk

Get fields

Enter field mapping

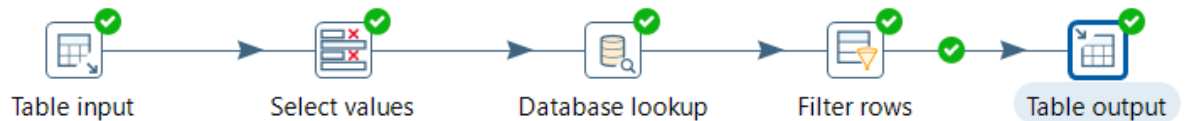
Help OK Cancel SQL

Setiap field dari stream dipetakan dengan kolom yang sesuai pada tabel Dim_Produk. Setelah proses load dijalankan, data yang sudah bersih dan sesuai struktur berhasil tersimpan ke dalam tabel Dim_Produk, sehingga dapat digunakan untuk mendukung pembentukan tabel fakta dan analisis data warehouse.

➤ Hasil Step Metrics

Execution Results													
Logging Execution History Step Metrics Performance Graph Metrics Preview data													
#	Stepname	Copynr	Read	Written	Input	Output	Updated	Rejected	Errors	Active	Time	Speed (r/s)	input/output
1	Table input	0	0	32951	32951	0	0	0	0	0 Finished	5.6s	5,885	-
2	Select values	0	32951	32951	0	0	0	0	0	0 Finished	8.2s	4,022	-
3	Database lookup	0	32951	32951	0	0	0	0	0	0 Finished	10.4s	3,157	-
4	Filter rows	0	32951	32951	0	0	0	0	0	0 Finished	10.4s	3,157	-
5	Table output	0	32951	32951	0	32951	0	0	0	0 Finished	10.6s	3,122	-

2. Tabel Dim_Pelanggan



➤ Extract

Proses Extract dilakukan menggunakan komponen Tabel Input dengan data yang diambil langsung dari tabel-tabel di database OLTP db_oltp_olist melalui koneksi bernama conn_oltp_olist. Berikut adalah konfigurasi Tabel Input pada Dim_Pelanggan:

Table input

Step name

Table input

Connection

conn_oltp_olist

Edit...

New...

Wizard...

SQL

Get SQL select statement...

```

SELECT DISTINCT
  customer_id,
  customer_zip_code_prefix,
  customer_city,
  customer_state
FROM olist_customers_dataset
WHERE customer_id IS NOT NULL;

```

Line 1 Column 0

Store column info in step

Enable lazy conversion

Replace variables in script?

Insert data from step

Execute for each row?

Limit size

0

Help

OK

Preview

Cancel

Query pada Table Input Dim_Pelanggan digunakan untuk mengambil data pelanggan dari tabel olist_customers_dataset. Query ini memilih kolom customer_id, customer_zip_code_prefix, customer_city, dan customer_state sesuai kebutuhan tabel Dim_Pelanggan. Penggunaan Select Distinct bertujuan agar data pelanggan tidak duplikat, sedangkan Where customer_id IS NOT NULL digunakan untuk memastikan hanya data pelanggan yang memiliki id valid yang diproses.

➤ Transform

Pada tahap transform, data yang telah diambil dari database OLTP diolah agar sesuai dengan struktur tabel tujuan pada data warehouse

a. Konfigurasi Select Values

Step name: **Select values**

Select & Alter Remove Meta-data

Fields :

#	Fieldname	Rename to	Length
1	customer_id	id_pelanggan	
2	customer_zip_code_prefix	kode_pos_pelanggan	
3	customer_city	kota_pelanggan	
4	customer_state	provinsi_pelanggan	

Get fields to select
Edit Mapping

Include unspecified fields, ☐

Help OK Cancel

Komponen Select Values digunakan untuk memilih field yang diperlukan serta menyesuaikan nama kolom dengan tabel tujuan.

b. Konfigurasi database lookup

Step name: **Database lookup**

Connection: conn_olap_olist Edit... New... Wizard...

Lookup schema: Browse...

Lookup table: dim_pelanggan Browse...

Enable cache? ☐

Cache size in rows (0=cache): 0

Load all data from table ☐

The key(s) to look up the value(s):

#	Table field	Comparator	Field1	Field2
1	id_pelanggan	=	id_pelanggan	

Values to return from the lookup table :

#	Field	New name	Default	Type
1	id_pelanggan	id_cek		None

Do not pass the row if the lookup fails ☐

Fail on multiple results? ☐

Order by:

Help OK Cancel Get Fields Get lookup fields

Database Lookup digunakan untuk mengecek apakah data sudah tersedia di tabel Dim_Pelanggan, sehingga dapat mencegah terjadinya duplikasi data.

c. konfigurasi filter rows

Step name:

Send 'true' data to step:

Send 'false' data to step:

The condition:

Filter Rows digunakan untuk menyaring data yang belum ada pada tabel Dim_Pelanggan, sehingga hanya data baru yang akan diteruskan ke proses load.

➤ Load

Step name:

Connection:

Target schema:

Target table:

Commit size:

Truncate table: ☐

Ignore insert errors: ☐

Specify database fields: ☒

Main options | Database fields

Fields to insert:

#	Table field	Stream field
1	id_pelanggan	id_pelanggan
2	kode_pos_pelanggan	kode_pos_pelanggan
3	kota_pelanggan	kota_pelanggan
4	provinsi_pelanggan	provinsi_pelanggan

Setiap field dari stream dipetakan dengan kolom yang sesuai pada tabel Dim_Pelanggan. Setelah proses load dijalankan, data yang sudah bersih dan sesuai struktur berhasil tersimpan ke dalam tabel Dim_Pelanggan, sehingga dapat digunakan untuk mendukung pembentukan tabel fakta dan analisis data warehouse.

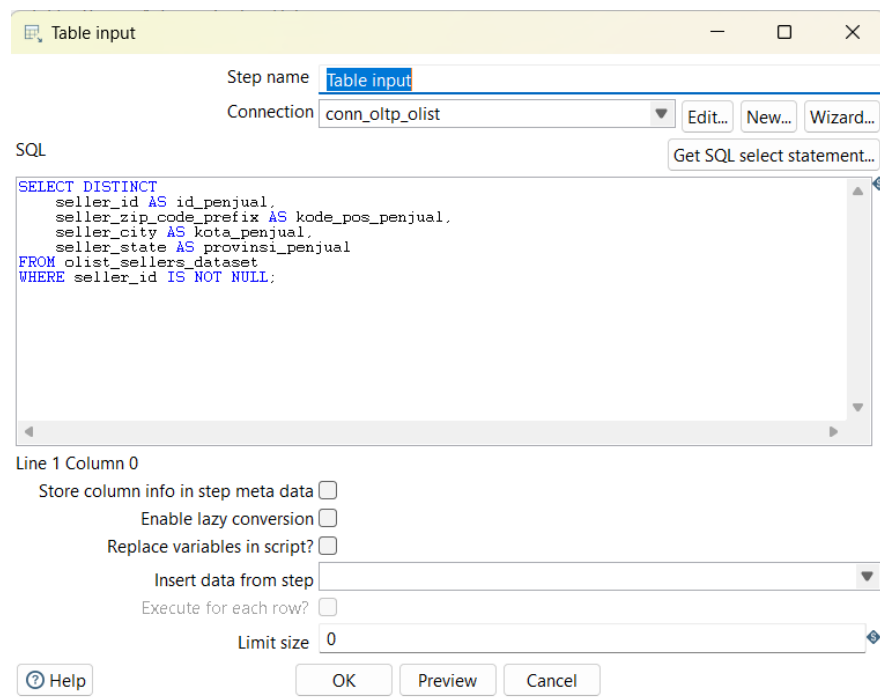
➤ Hasil Step Metrics

Execution Results												
Logging Execution History Step Metrics Performance Graph Metrics Preview data												
#	Stepname	Copynr	Read	Written	Input	Output	Updated	Rejected	Errors	Active	Time	Speed (r/s)
1	Table input	0	0	99441	99441	0	0	0	0	Finished	51.0s	1,948
2	Select values	0	99441	99441	0	0	0	0	0	Finished	56.8s	1,752
3	Database lookup	0	99441	99441	0	0	0	0	0	Finished	1mn 1s	1,607
4	Filter rows	0	99441	99441	0	0	0	0	0	Finished	1mn 1s	1,607
5	Table output	0	99441	99441	0	99441	0	0	0	Finished	1mn 2s	1,603

3. Tabel Dim_Penjual

➤ Extract

Proses Extract dilakukan menggunakan komponen Tabel Input dengan data yang diambil langsung dari tabel-tabel di database OLTP db_oltpolist melalui koneksi bernama conn_oltpolist. Berikut adalah konfigurasi Tabel Input pada Dim_Penjual:



Query ini memilih kolom seller_id, seller_zip_code_prefix, seller_city, dan seller_state, lalu langsung memberi alias sesuai struktur tabel Dim_Penjual, yaitu id_penjual, kode_pos_penjual, kota_penjual, dan provinsi_penjual. Penggunaan Select Distinct bertujuan agar data penjual tidak duplikat, sedangkan Where seller_id IS NOT NULL memastikan hanya data penjual yang memiliki id valid yang diproses.

➤ Transform

Pada tahap transform, data yang telah diambil dari database OLTP diolah agar sesuai dengan struktur tabel tujuan pada data warehouse

a. Konfigurasi Select Values

#	Fieldname	Rename to	Length	Precision
1	id_penjual	StreamIdPenjual		
2	kode_pos_penjual	StreamKodePosPenjualan		
3	kota_penjual	StreamKotaPenjual		
4	provinsi_penjual	StreamProvinsiPenjual		

Komponen Select Values digunakan untuk memilih field yang diperlukan serta menyesuaikan nama kolom dengan tabel tujuan.

b. Konfigurasi database lookup

#	Table field	Comparator	Field1	Field2
1	id_penjual	=	StreamIdPenjual	

#	Field	New name	Default	Type
1	id_penjual			String

Database Lookup digunakan untuk mengecek apakah data sudah tersedia di tabel Dim_Pelanggan, sehingga dapat mencegah terjadinya duplikasi data.

c. Konfigurasi Filter Rows

Step name: **Filter rows**

Send 'true' data to step: **Table output**

Send 'false' data to step:

The condition:

IS NULL

Filter Rows digunakan untuk menyaring data yang belum ada pada tabel Dim_Penjual, sehingga hanya data baru yang akan diteruskan ke proses load.

➤ Load

Step name: **Table output**

Connection: **conn_olap_olist**

Target schema: **dw_penjualan_olist**

Target table: **dim_penjual**

Commit size: **1000**

Truncate table: ☐

Ignore insert errors: ☐

Specify database fields: ☒

Main options | Database fields

Fields to insert:

#	Table field	Stream field
1	id_penjual	StreamIdPenjual
2	kode_pos_penjual	StreamKodePosPenjualan
3	kota_penjual	StreamKotaPenjual
4	provinsi_penjual	StreamProvinsiPenjual

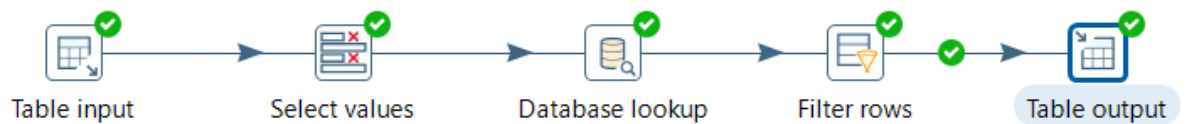
Buttons: Get fields, Enter field mapping, OK, Cancel, SQL, Help

Setiap field dari stream dipetakan dengan kolom yang sesuai pada tabel Dim_Penjual. Setelah proses load dijalankan, data yang sudah bersih dan sesuai struktur berhasil tersimpan ke dalam tabel Dim_Penjual, sehingga dapat digunakan untuk mendukung pembentukan tabel fakta dan analisis data warehouse.

➤ Hasil Step Metrics

Execution Results														
Logging Execution History Step Metrics Performance Graph Metrics Preview data														
#	Stepname	Copynr	Read	Written	Input	Output	Updated	Rejected	Errors	Active	Time	Speed (r/s)	input/output	
1	Table input	0	0	3095	3095	0	0	0	0	0	0.1s	42,986	-	-
2	Select values	0	3095	3095	0	0	0	0	0	Finished	0.1s	42,986	-	-
3	Database lookup	0	3095	3095	0	0	0	0	0	Finished	2.6s	1,193	-	-
4	Filter rows	0	3095	3095	0	0	0	0	0	Finished	2.6s	1,175	-	-
5	Table output	0	3095	3095	0	3095	0	0	0	Finished	3.0s	1,023	-	-

4. Tabel Dim_Waktu



➤ Extract

Proses Extract dilakukan menggunakan komponen Tabel Input dengan data yang diambil langsung dari tabel-tabel di database OLTP db_oltp_olist melalui koneksi bernama conn_oltp_olist. Berikut adalah konfigurasi Tabel Input pada Dim_Waktu:

Table input

Step name: Table input

Connection: conn_oltp_olist

SQL

```
SELECT DISTINCT
  DATE(order_purchase_timestamp) AS tanggal_lengkap,
  EXTRACT(DAY FROM order_purchase_timestamp) AS hari,
  EXTRACT(MONTH FROM order_purchase_timestamp) AS bulan,
  EXTRACT(YEAR FROM order_purchase_timestamp) AS tahun,
  CONCAT('Q', QUARTER(order_purchase_timestamp)) AS kuartal
FROM olist_orders_dataset
WHERE order_purchase_timestamp IS NOT NULL
ORDER BY tanggal_lengkap ASC;
```

Line 1 Column 0

Store column info in step meta ☐

Enable lazy conversion ☐

Replace variables in script? ☐

Insert data from step

Execute for each row? ☐

Limit size: 0

Help OK Preview Cancel

Query pada Table Input Dim_Waktu digunakan untuk mengambil data tanggal transaksi dari tabel olist_orders_dataset. Query ini memakai Select Distinct agar tanggal yang masuk ke Dim_Waktu tidak duplikat. Kolom order_purchase_timestamp diolah menjadi beberapa atribut waktu, yaitu tanggal_lengkap, hari, bulan, tahun, dan kuartal. Selain itu, kondisi Where order_purchase_timestamp IS NOT NULL digunakan agar hanya data transaksi yang memiliki tanggal valid yang diproses. Hasil query kemudian diurutkan berdasarkan tanggal_lengkap secara naik atau dari tanggal paling awal.

➤ Transform

Pada tahap transform, data yang telah diambil dari database OLTP diolah agar sesuai dengan struktur tabel tujuan pada data warehouse

a. Konfigurasi Select Values

#	Fieldname	Rename to	Length	Precision
1	tanggal_lengkap	streamTglLengkap		
2	hari	streamHari		
3	bulan	streamBulan		
4	tahun	streamTahun		
5	kuartal	streamKuartal		

Komponen Select Values digunakan untuk memilih field yang diperlukan serta menyesuaikan nama kolom dengan tabel tujuan.

b. Konfigurasi database lookup

#	Table field	Comparator	Field1	Field2
1	tanggal_lengkap	=	streamTglLengkap	

#	Field	New name	Default	Type
1	id_waktu	id_dimWaktu		Integer

Database Lookup digunakan untuk mengecek apakah data sudah tersedia di tabel Dim_Waktu, sehingga dapat mencegah terjadinya duplikasi data.

c. Konfigurasi Filter Rows

Step name: Filter rows

Send 'true' data to step: Table output

Send 'false' data to step:

The condition:

id_dimWaktu IS NULL

Filter Rows digunakan untuk menyaring data yang belum ada pada tabel Dim_Waktu, sehingga hanya data baru yang akan diteruskan ke proses load.

➤ Load

Step name: Table output

Connection: conn_olap_olist

Target schema: dw_penjualan_olist

Target table: dim_waktu

Commit size: 1000

Truncate table: ☐

Ignore insert errors: ☐

Specify database fields: ☒

Main options | Database fields

Fields to insert:

#	Table field	Stream field
1	hari	streamHari
2	bulan	streamBulan
3	tahun	streamTahun
4	kuartal	streamQuartal
5	tanggal_lengkap	streamTglLengkap

Get fields

Enter field mapping

Help OK Cancel SQL

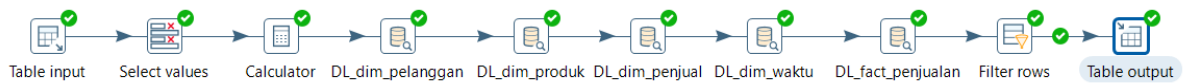
Setiap field dari stream dipetakan dengan kolom yang sesuai pada tabel Dim_Waktu. Setelah proses load dijalankan, data yang sudah bersih dan sesuai struktur berhasil tersimpan ke dalam tabel Dim_Waktu, sehingga dapat digunakan untuk mendukung pembentukan tabel fakta dan analisis data warehouse.

➤ Hasil Step Metrics

Execution Results													
Logging Execution History Step Metrics Performance Graph Metrics Preview data													
#	Stepname	Copynr	Read	Written	Input	Output	Updated	Rejected	Errors	Active	Time	Speed (r/s)	input/output
1	Table input	0	0	634	634	0	0	0	0	Finished	0.1s	6,745	-
2	Select values	0	634	634	0	0	0	0	0	Finished	0.1s	5,981	-
3	Database lookup	0	634	634	0	0	0	0	0	Finished	0.2s	3,409	-
4	Filter rows	0	634	634	0	0	0	0	0	Finished	0.2s	3,170	-
5	Table output	0	634	634	0	634	0	0	0	Finished	0.3s	2,164	-

3.4 Implementasi ETL Pada Tabel Fakta

1. Tabel Fact_Penjualan



➤ Extract

Proses Extract dilakukan menggunakan komponen Tabel Input dengan data yang diambil langsung dari tabel-tabel di database OLTP db_oltp_olist melalui koneksi bernama conn_oltp_olist. Berikut adalah konfigurasi Tabel Input pada Fact_Penjualan:

Query pada Table Input Fact_Penjualan digunakan untuk mengambil data transaksi penjualan dari dua tabel sumber, yaitu olist_orders_dataset dan olist_order_items_dataset. Kedua tabel tersebut digabungkan menggunakan JOIN berdasarkan kolom order_id, karena data pesanan seperti customer_id dan order_purchase_timestamp berada pada tabel orders, sedangkan data detail transaksi seperti product_id, seller_id, price, dan freight_value berada pada tabel order items. Kolom pada query ini digunakan untuk membentuk tabel Fact_Penjualan. Selain itu, query menggunakan kondisi IS NOT NULL untuk memastikan data yang diambil tidak kosong.

➤ Transform

Pada tahap transform, data yang telah diambil dari database OLTP diolah agar sesuai dengan struktur tabel tujuan pada data warehouse

a. Konfigurasi Select Values

#	Fieldname	Rename to	Length	Precision
1	order_id	streamIdPesanan		
2	customer_id	streamIdPelanggan		
3	product_id	streamIdProduk		
4	seller_id	streamIdPenjual		
5	order_purchase_timestamp	streamIdWaktu		
6	price	streamHargaBarang		
7	freight_value	streamOngkosKirim		

Konfigurasi Select Values digunakan untuk memilih kolom-kolom yang dibutuhkan dari hasil input data. Selain itu, pada tahap ini juga dilakukan penyesuaian nama kolom agar sesuai dengan struktur tabel Fact_Penjualan,

b. Konfigurasi Calculator

#	New field	Calculation	Field A	Field B	Field C	Value type	Length	Precision	Remove	Conversion ma
1	tahun	Year of date A	streamIdWaktu			Integer			N	
2	bulan	Month of date A	streamIdWaktu			Integer			N	
3	hari	Day of month of date A	streamIdWaktu			Integer			N	

Konfigurasi Calculator digunakan untuk membuat kolom baru yang dibutuhkan pada tabel fakta. Proses ini, kolom order_purchase_timestamp diolah untuk menghasilkan id_waktu dengan format YYYYMMDD.

c. Konfigurasi Database Lookup ke Dim_Pelanggan

The screenshot shows the 'Database lookup' configuration window for a step named 'DL_dim_pelanggan'. The connection is 'conn_olap_olist', the schema is 'dw_penjualan_olist', and the lookup table is 'dim_pelanggan'. The 'Enable cache?' checkbox is unchecked, and the 'Cache size in rows' is set to 0. The 'Load all data from table' checkbox is also unchecked. Below these settings, there is a table for 'The key(s) to look up the value(s):' with one row: #1, Table field: id_pelanggan, Comparator: =, Field1: streamIdPelanggan, and Field2 is empty. Another table, 'Values to return from the lookup table:', has one row: #1, Field: id_pelanggan, New name is empty, Default is empty, and Type is None. At the bottom, there are checkboxes for 'Do not pass the row if the lookup fails' and 'Fail on multiple results?', both unchecked, and an 'Order by' field. The window has buttons for Help, OK, Cancel, Get Fields, and Get lookup fields.

#	Table field	Comparator	Field1	Field2
1	id_pelanggan	=	streamIdPelanggan	

#	Field	New name	Default	Type
1	id_pelanggan			None

Konfigurasi Database Lookup ke Dim_Pelanggan digunakan untuk mengecek apakah data pelanggan dari kolom id_pelanggan sudah tersedia di tabel Dim_Pelanggan.

d. Konfigurasi Database Lookup ke Dim_Produk

The screenshot shows the 'Database lookup' configuration window for a step named 'DL_dim_produk'. The connection is 'conn_olap_olist', the schema is 'dw_penjualan_olist', and the lookup table is 'dim_produk'. The 'Enable cache?' checkbox is unchecked, and the 'Cache size in rows' is set to 0. The 'Load all data from table' checkbox is also unchecked. Below these settings, there is a table for 'The key(s) to look up the value(s):' with one row: #1, Table field: id_produk, Comparator: =, Field1: streamIdProduk, and Field2 is empty. Another table, 'Values to return from the lookup table:', has one row: #1, Field: id_produk, New name is empty, Default is empty, and Type is None. At the bottom, there are checkboxes for 'Do not pass the row if the lookup fails' and 'Fail on multiple results?', both unchecked, and an 'Order by' field. The window has buttons for Help, OK, Cancel, Get Fields, and Get lookup fields.

#	Table field	Comparator	Field1	Field2
1	id_produk	=	streamIdProduk	

#	Field	New name	Default	Type
1	id_produk			None

Konfigurasi Database Lookup ke Dim_Produk digunakan untuk mencocokkan kolom id_produk dengan data yang ada pada tabel Dim_Produk untuk memastikan bahwa setiap produk yang masuk ke tabel fakta sudah terdaftar pada tabel dimensi produk.

e. Konfigurasi Database Lookup ke Dim_Penjual

The screenshot shows the 'Database lookup' configuration window for a step named 'DL_dim_penjual'. The configuration includes the following details:

- Step name:** DL_dim_penjual
- Connection:** conn_olap_olist
- Lookup schema:** dw_penjualan_olist
- Lookup table:** dim_penjual
- Enable cache?** ☐
- Cache size in rows (0=cache):** 0
- Load all data from table:** ☐

The key(s) to look up the value(s):

#	Table field	Comparator	Field1	Field2
1	id_penjual	=	streamIdPenjual	

Values to return from the lookup table :

#	Field	New name	Default	Type
1	id_penjual			None

Do not pass the row if the lookup fails: ☐
Fail on multiple results?: ☐
Order by:

Buttons at the bottom: Help, OK, Cancel, Get Fields, Get lookup fields.

Konfigurasi Database Lookup ke Dim_Penjual digunakan untuk memeriksa apakah id_penjual dari data transaksi sudah ada di tabel Dim_Penjual.

f. Konfigurasi Database Lookup ke Dim_Waktu

The screenshot shows the 'Database lookup' configuration window for a step named 'DL_dim_waktu'. The configuration includes the following details:

- Step name:** DL_dim_waktu
- Connection:** conn_olap_olist
- Lookup schema:** dw_penjualan_olist
- Lookup table:** dim_waktu
- Enable cache?** ☐
- Cache size in rows (0=cache):** 0
- Load all data from table:** ☐

The key(s) to look up the value(s):

#	Table field	Comparator	Field1	Field2
1	hari	=	hari	
2	bulan	=	bulan	
3	tahun	=	tahun	

Values to return from the lookup table :

#	Field	New name	Default	Type
1	id_waktu	lookup_waktu		None

Do not pass the row if the lookup fails: ☐
Fail on multiple results?: ☐
Order by:

Buttons at the bottom: Help, OK, Cancel, Get Fields, Get lookup fields.

Konfigurasi Database Lookup ke Dim_Waktu digunakan untuk mencocokkan id_waktu hasil dari proses Calculator dengan data yang ada pada tabel Dim_Waktu. Ini memastikan bahwa setiap transaksi memiliki referensi waktu yang valid, sehingga data dapat dianalisis berdasarkan tanggal, bulan, tahun, atau kuartal.

g. Konfigurasi Database Lookup ke Fact_Penjualan

Database lookup

Step name: **DL_fact_penjualan**

Connection: **conn_olap_olist** [Edit...] [New...] [Wizard...]

Lookup schema: **dw_penjualan_olist** [Browse...]

Lookup table: **fact_penjualan** [Browse...]

Enable cache? ☐

Cache size in rows (0=cache): **0**

Load all data from table: ☐

The key(s) to look up the value(s):

#	Table field	Comparator	Field1	Field2
1	id_pesanan	=	streamIdPesanan	
2	id_pelanggan	=	streamIdPelanggan	
3	id_produk	=	streamIdProduk	
4	id_penjual	=	streamIdPenjual	
5	id_waktu	=	lookup_waktu	
6	harna_barang	=	streamHarnaBarang	

Values to return from the lookup table:

#	Field	New name	Default	Type
1	id_fakta			None

Do not pass the row if the lookup fails: ☐

Fail on multiple results?: ☐

Order by:

[OK] [Cancel] [Get Fields] [Get lookup fields]

Konfigurasi Database Lookup ke Fact_Penjualan digunakan untuk mengecek apakah data transaksi yang akan dimasukkan sudah pernah ada di tabel Fact_Penjualan atau belum.

h. Filter Rows

Filter rows

Step name: **Filter rows**

Send 'true' data to step: **Table output**

Send 'false' data to step:

The condition:

☐ +

id_pelanggan IS NOT NULL

AND

id_produk IS NOT NULL

AND

id_penjual IS NOT NULL

AND

lookup_waktu IS NOT NULL

AND

id_fakta IS NULL

[Help] [OK] [Cancel]

Activate Windows
Go to Settings to activate Windows

Konfigurasi Filter Rows digunakan untuk menyaring data. Data yang memiliki referensi lengkap ke tabel dimensi dan belum terdapat pada tabel Fact_Penjualan akan diteruskan ke tahap load. Sementara itu, data yang tidak valid atau sudah ada sebelumnya tidak dimasukkan lagi ke tabel fakta.

➤ Load

Table output

Step name: Table output

Connection: conn_olap_olist

Target schema: dw_penjualan_olist

Target table: fact_penjualan

Commit size: 1000

Truncate table: ☐

Ignore insert errors: ☐

Specify database fields: ☒

Main options | Database fields

Fields to insert:

#	Table field	Stream field		
1	id_pesanan	streamIdPesanan		
2	id_pelanggan	streamIdPelanggan		
3	id_produk	streamIdProduk		
4	id_penjual	streamIdPenjual		
5	id_waktu	lookup_waktu		
6	harga_barang	streamHargaBarang		
7	ongkos_kirim	streamOngkosKirim		

Get fields

Enter field mapping

Help OK Cancel SQL

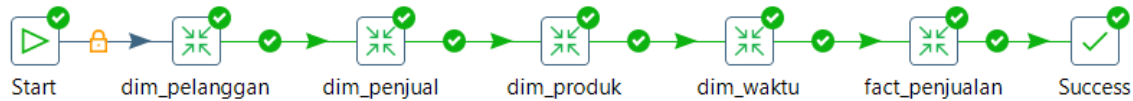
Pada tahap Load, data hasil transformasi dimasukkan ke dalam tabel Fact_Penjualan. Proses load dilakukan setelah seluruh tabel dimensi, selesai dimuat. Hal ini dilakukan karena tabel Fact_Penjualan memiliki foreign key yang mengacu ke semua tabel dimensi tersebut. Data yang dimuat berisi informasi transaksi penjualan, seperti pesanan, pelanggan, produk, penjual, waktu transaksi, harga barang, dan ongkos kirim. Dengan proses load ini, tabel Fact_Penjualan menjadi tabel utama yang digunakan untuk analisis KPI penjualan, seperti total penjualan, jumlah transaksi, dan total ongkos kirim.

➤ Hasil Step Metrics

#	Stepname	Copynr	Read	Written	Input	Output	Updated	Rejected	Errors	Active	Time	Speed (r/s)	input/output
1	Table input	0	0	112650	112650	0	0	0	0	Finished	45.9s	2,456	-
2	Select values	0	112650	112650	0	0	0	0	0	Finished	54.2s	2,077	-
3	Calculator	0	112650	112650	0	0	0	0	0	Finished	1mn 2s	1,802	-
4	DL_dim_pelanggan	0	112650	112650	112650	0	0	0	0	Finished	1mn 10s	1,589	-
5	DL_dim_produk	0	112650	112650	112650	0	0	0	0	Finished	1mn 18s	1,430	-
6	DL_dim_penjual	0	112650	112650	112650	0	0	0	0	Finished	1mn 26s	1,306	-
7	DL_dim_waktu	0	112650	112650	112650	0	0	0	0	Finished	1mn 33s	1,207	-
8	DL_fact_penjualan	0	112650	112650	0	0	0	0	0	Finished	1mn 33s	1,206	-
9	Filter rows	0	112650	112650	0	0	0	0	0	Finished	1mn 33s	1,206	-
10	Table output	0	112650	112650	0	112650	0	0	0	Finished	1mn 33s	1,202	-

3.5 Implementasi Jobs

Job ETL dirancang agar proses pemuatan data ke dalam data warehouse berjalan secara terstruktur dan menjaga integritas data antar tabel. Urutan eksekusi dimulai dari pemuatan tabel dimensi, kemudian dilanjutkan dengan pemuatan tabel fakta. Adapun urutan proses pada Job ETL adalah sebagai berikut:



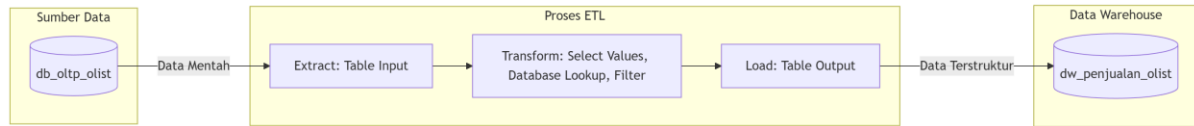
1. Start: Titik awal eksekusi job.
2. dim_pelanggan: Memuat data master pelanggan ke data warehouse.
3. dim_penjual: Memuat data master penjual.
4. dim_produk: Memuat data master produk.
5. dim_waktu: Memuat data referensi waktu transaksi.
6. fact_penjualan: Memuat data transaksi penjualan yang telah terhubung dengan seluruh tabel dimensi.
7. Success: Menandakan seluruh rangkaian proses ETL telah selesai dijalankan dengan hasil sukses.

Penggunaan Job memberikan beberapa keuntungan, seperti otomatisasi proses ETL, pengurangan kesalahan manual saat menjalankan transformasi satu per satu, serta mempermudah pemeliharaan pipeline ketika terjadi perubahan pada struktur data atau kebutuhan bisnis. Berdasarkan hasil Job Metrics yang kami peroleh, seluruh tahapan berjalan dengan status Success tanpa adanya kesalahan (errors) pada tiap job entry seperti gambar berikut:

Execution Results						
Logging History Job metrics Metrics						
Job / Job Entry	Comment	Result	Reason	Filename	Nr	Log date
Job: master_etl_job						
Start	Start of job execution		start			2026/06/04 21:...
Start	Job execution finished	Success			0	2026/06/04 21:...
dim_pelanggan	Start of job execution		Followed unconditional link	D:\data warehouse\Project...		2026/06/04 21:...
dim_pelanggan	Job execution finished	Success		D:\data warehouse\Project...	1	2026/06/04 21:...
dim_penjual	Start of job execution		Followed link after success	D:\data warehouse\Project...		2026/06/04 21:...
dim_penjual	Job execution finished	Success		D:\data warehouse\Project...	2	2026/06/04 21:...
dim_produk	Start of job execution		Followed link after success	D:\data warehouse\Project...		2026/06/04 21:...
dim_produk	Job execution finished	Success		D:\data warehouse\Project...	3	2026/06/04 21:...
dim_waktu	Start of job execution		Followed link after success	D:\data warehouse\Project...		2026/06/04 21:...
dim_waktu	Job execution finished	Success		D:\data warehouse\Project...	4	2026/06/04 21:...
fact_penjualan	Start of job execution		Followed link after success	D:\data warehouse\Project...		2026/06/04 21:...
fact_penjualan	Job execution finished	Success		D:\data warehouse\Project...	5	2026/06/04 21:...
Success	Start of job execution		Followed link after success			2026/06/04 21:...
Success	Job execution finished	Success			5	2026/06/04 21:...
Job: master_etl_job	Job execution finished	Success	finished		5	2026/06/04 21:...

3.6 Alur Pipeline ETL

Pipeline ETL yang diterapkan pada proyek data warehouse Olist terdiri dari tiga tahapan utama, yaitu Extract, Transform, dan Load. Proses ini bertujuan untuk mengubah data operasional yang berasal dari database OLTP menjadi data yang terstruktur untuk kebutuhan analisis.



1. Extract

Tahap Extract merupakan proses pengambilan data dari database db_oltp_olist. Data yang diekstraksi berasal dari beberapa tabel utama, yaitu:

- olist_customers_dataset
- olist_sellers_dataset
- olist_products_dataset
- product_category_name_translation
- olist_orders_dataset
- olist_order_items_dataset

Proses ekstraksi dilakukan menggunakan komponen Table Input pada Pentaho PDI dengan query SQL yang dilengkapi dengan fungsi filtering seperti IS NOT NULL dan DISTINCT untuk memastikan hanya data yang bersih dan unik yang akan diproses sesuai kebutuhan masing-masing tabel dimensi dan fakta.

2. Transform

Setelah data berhasil diekstraksi, dilakukan proses transformasi untuk menyesuaikan struktur data dengan rancangan star schema yang telah dibuat. Beberapa proses transformasi yang dilakukan antara lain:

- Select Values untuk pemetaan kolom.
- Calculator untuk pembentukan atribut waktu.
- Database Lookup (untuk pengecekan integritas data atau duplikasi).
- Filter Rows untuk memastikan hanya data yang valid dan baru yang akan diteruskan ke tahap selanjutnya

Tahap transformasi memastikan bahwa data yang dimuat ke data warehouse telah memiliki format yang konsisten dan sesuai dengan kebutuhan analisis.

3. Load

Tahap Load merupakan proses memasukkan data hasil transformasi ke dalam database data warehouse yaitu dw_penjualan_olist. Proses loading dilakukan secara bertahap dengan urutan:

- dim_pelanggan
- dim_penjual
- dim_produk
- dim_waktu
- fact_penjualan

Data dimuat menggunakan komponen Table Output pada Pentaho PDI. Setelah proses loading selesai, seluruh data telah tersimpan pada tabel dimensi dan fakta sehingga dapat digunakan untuk pembuatan laporan maupun analisis KPI bisnis.

3.7 Tools yang Digunakan

Dalam pengembangan data warehouse ini digunakan beberapa perangkat lunak yang saling terintegrasi untuk mendukung proses ETL, penyimpanan data, serta pengelolaan proyek. Berikut adalah beberapa tools yang digunakan:

Tools	Fungsi
Pentaho Data Integration (PDI)/Spoon	Membuat dan menjalankan proses ETL
MySQL	Penyimpanan database OLTP dan Data Warehouse
phpMyAdmin	Administrasi database dan validasi data
GitHub	Version control dan penyimpanan proyek
Microsoft Word	Penyusunan dokumentasi dan laporan

3. Dim_Produk (32951 baris)

Showing rows 0 - 24 (32951 total, Query took 0.0026 seconds.)

`SELECT * FROM `dim_produk``

☐ Profiling [\[Edit inline \]](#) [\[Edit \]](#) [\[Explain SQL \]](#) [\[Create PHP code \]](#) [\[Refresh \]](#)

1 > >> | Number of rows: 25 | Filter rows: Search this table | Sort by key: None

Extra options

				id_produk	kategori_produk	berat_produk_gram	panjang_produk_cm
☐	Edit	Copy	Delete	00066f42aeeb9f3007548bb9d3f33c38	perfumery	300	20
☐	Edit	Copy	Delete	00088930e925c41fd95ebfe695fd2655	auto	1225	55
☐	Edit	Copy	Delete	0009406fd7479715e4bef61dd91f2462	bed_bath_table	300	45
☐	Edit	Copy	Delete	000b8f95fcb9e0096488278317764d19	housewares	550	19
☐	Edit	Copy	Delete	000d9be29b5207b54e86aa1b1ac54872	watches_gifts	250	22
☐	Edit	Copy	Delete	0011c512eb256aa0dbbb544d8dfcf6e	auto	100	16
☐	Edit	Copy	Delete	00126f27c813603687e6ce486d909d01	cool_stuff	700	25
☐	Edit	Copy	Delete	001795ec6f1b187d37335e1c4704762e	consoles_games	600	30
☐	Edit	Copy	Delete	001b237c0e9bb435f2e54071129237e9	bed_bath_table	6000	40
☐	Edit	Copy	Delete	001b72dfd63e9833e8c02742adf472e3	furniture_decor	600	26
☐	Edit	Copy	Delete	001c5d71ac6ad696d22315953758fa04	bed_bath_table	1800	47
☐	Edit	Copy	Delete	00210e41887c2a8ef9f791ebc780cc36	health_beauty	300	30
☐	Edit	Copy	Delete	002159fe700ed3521f46cfcf6e941c76	fashion_shoes	1850	36
☐	Edit	Copy	Delete	0021a87d4997a48b6cef1665602be0f5	computers_accessories	650	25
☐	Edit	Copy	Delete	00250175f79f584c14ab5cecd80553cd	housewares	750	30
☐	Edit	Copy	Delete	002552c0663708129c0019cc97552d7d	cool_stuff	355	22
☐	Edit	Copy	Delete	002959d7a0b0990fe2d69988affc80	furniture_decor	2600	105
☐	Edit	Copy	Delete	003099741b70c7b5cf4e4a0ad7ef85	cool_stuff	600	40

4. Dim_Waktu (634 baris)

Showing rows 0 - 24 (634 total, Query took 0.0010 seconds.)

`SELECT * FROM `dim_waktu``

☐ Profiling [\[Edit inline \]](#) [\[Edit \]](#) [\[Explain SQL \]](#) [\[Create PHP code \]](#) [\[Refresh \]](#)

1 > >> | Number of rows: 25 | Filter rows: Search this table | Sort

Extra options

					id_waktu	tanggal_lengkap	hari	bulan	tahun	kuartal
☐	Edit	Copy	Delete	1	2016-09-04		4	9	2016	Q3
☐	Edit	Copy	Delete	2	2016-09-05		5	9	2016	Q3
☐	Edit	Copy	Delete	3	2016-09-13		13	9	2016	Q3
☐	Edit	Copy	Delete	4	2016-09-15		15	9	2016	Q3
☐	Edit	Copy	Delete	5	2016-10-02		2	10	2016	Q4
☐	Edit	Copy	Delete	6	2016-10-03		3	10	2016	Q4
☐	Edit	Copy	Delete	7	2016-10-04		4	10	2016	Q4
☐	Edit	Copy	Delete	8	2016-10-05		5	10	2016	Q4
☐	Edit	Copy	Delete	9	2016-10-06		6	10	2016	Q4
☐	Edit	Copy	Delete	10	2016-10-07		7	10	2016	Q4
☐	Edit	Copy	Delete	11	2016-10-08		8	10	2016	Q4
☐	Edit	Copy	Delete	12	2016-10-09		9	10	2016	Q4
☐	Edit	Copy	Delete	13	2016-10-10		10	10	2016	Q4
☐	Edit	Copy	Delete	14	2016-10-22		22	10	2016	Q4
☐	Edit	Copy	Delete	15	2016-12-23		23	12	2016	Q4
☐	Edit	Copy	Delete	16	2017-01-05		5	1	2017	Q1
☐	Edit	Copy	Delete	17	2017-01-06		6	1	2017	Q1
☐	Edit	Copy	Delete	18	2017-01-07		7	1	2017	Q1

5. Fact_Penjualan (112650 baris)

vs 0 - 24 (111496 total, Query took 0.0036 seconds.)

fact_penjualan

it inline | Edit | Explain SQL | Create PHP code | Refresh

> >> | Number of rows: 25 | Filter rows: Search this table | Sort by key: None

	id_fakta	id_pesanan	id_pelanggan	id_produk	id_penjualan	id_waktu	harga_barang	ongkos_kirim
Copy Delete	1	00010242fe8c5a6d1ba2dd792cb16214	3ce436f183e69e07877b285a838db11a	4244733e06e7ecb4970a6e2683c13e61	48436dade18ac8b2bce089ec2a041202	267	58.90	13.29
Copy Delete	2	000187712f0320c557190d7a144bdd3	f6dd3ec061db4e3987629f6b26e5cce	e5f2d52b02189ee55865ca93d33a8f	dd7ddc04e1b6c2c614352b383efe2d36	127	239.90	19.93
Copy Delete	3	000229ec398224ef8ca0657da4fc703e	6489ae5e4333f3693df5ad4372dab6d3	c777355d18b72b67abbef9df44f00fd	5b51032eddd242adc84c38acab88f23d	390	199.00	17.87
Copy Delete	4	00024acbf08a9daa1e931b038114c75	d4eb9395c8c0431ee92fce09960c5a06	7634da152a4610f1595efa32f14722fc	9d7a1d34a5052409009425275ba1c2b4	596	12.99	12.79
Copy Delete	5	00042b26c59d7ce9dfab4e5b64fd9	58dbdb0c2d70206bf40e62cd34e84d795	ac6c3623068f30de03045865e4e10089	df560393f3a51e74553ab94004ba5c87	46	199.90	18.14
Copy Delete	6	00048cc3ae777c65dbb7d2a0634bc1ea	816cbea969fe5b689b39cfc97a506742	ef92defde845ab8450f9d70c526ef70f	6426d21aca402a131fc0a5d0960a3c90	146	21.90	12.69
Copy Delete	7	00054e8431b9d7675808bcb819b4a32	32e2e6ab09e778d99bf2e0ecd4898718	8d4f2bb7e93e6710a28f34fa83ee7d28	7040e82f899a04d1b434b795a43b4617	355	19.90	11.85
Copy Delete	8	000576fe3919847cbb9d288c5617fa6	9ed5e522d9dd85b4af4a077526d8117	557d850972a7d6f792d18ae1400d9b6	5996cddab893a4652a15592b58ab8db	561	810.00	70.75
Copy Delete	9	0005a1a1728c9d785b8e2b08b904576c	16150771df4776261284213b89c304e	310ae3c140f94b03219ad0adc3c778f	a416b6a846a11724393025641d4edd5e	454	145.95	11.65
Copy Delete	10	0005f0442c8953cd1d21e1fb923495	351d3cb2cee3c7fd0af6186c82d21d3	4535b0e1091c278d1d193e5a1d63b39f	ba143b05d0110f0dc71ad71b4466ce92	559	53.99	11.40
Copy Delete	11	00061f2a7bc09da83e415a52dc8a4af1	c6fc061d86fab1e2b2eac259bac71a49	d63c1011f49d98b976c352955b1c4bea	cc419e0650a3c5ba77189a1882b7556a	459	59.99	8.88
Copy Delete	12	00063b381e2406b52ad429470734ebd5	6a899e5865de6549a58d2c5845e5604	1177554ea93259a5b282f24e33f65ab6	8602a61d680a10a82cceedad099ea3d	584	45.00	12.98
Copy Delete	13	0006ce9bd1a64e59a68b2c340b065a7	5d178120c29c61749ea95bac23cb825	99a4788cb24856965c36a24e339b0058	4a3ca9315b744ce9f8e9374361493884	581	74.00	23.32
Copy Delete	14	0008288aa43d2a3f00fcb17cd7d8719	2355af7c75e7c98b43a87ba7f210dc5	368c6c730842d78016ad823897a372db	1f50f920176fa81dad994f9023523100	420	49.90	13.37
Copy Delete	15	0008288aa43d2a3f00fcb17cd7d8719	2355af7c75e7c98b43a87ba7f210dc5	368c6c730842d78016ad823897a372db	1f50f920176fa81dad994f9023523100	420	49.90	13.37
Copy Delete	16	0009792311464db532f765df7b182ae	2a30c97868e81df7c17a8b14447aeaba	8cab9bac59158715e0d70a36c807415	530ec6109d11eaaf87999465c6afe01	602	99.90	27.65
Copy Delete	17	0009ca417916a706d71784483a5d643	8a250edc40ebc5c3940ebc940f16a7eb	3f27ac8e999df3d300ec4a5d8c5c0b2	fcfbace8bcc92f75707dc0f01a27d269	481	639.00	11.34
Copy Delete	18	000a2e2f50bd2f8d70f584c5a2d4d	ff5160ae5838d07fac9fec88962f188d	4fa33915031a80de03dd1d3e8fb27f01	fe20332dah1a61af8794248c8196595c9	507	144.00	8.77

4.2 Analisis Key Performance Indicators (KPI)

Analisis Key Performance Indicators atau KPI dilakukan untuk mengubah data transaksi yang tersimpan dalam data warehouse menjadi informasi yang lebih mudah dipahami. Analisis ini memanfaatkan tabel Fact_Penjualan sebagai sumber utama data transaksi serta tabel dimensi sebagai sumber informasi pendukung.

1. Tren Penjualan Bulanan dan Volume Transaksi

```
1 SELECT
2     dw.tahun,
3     dw.bulan,
4     ROUND(SUM(fp.harga_barang), 2) AS total_penjualan,
5     COUNT(fp.id_fakta) AS jumlah_item_terjual,
6     COUNT(DISTINCT fp.id_pesanan) AS jumlah_pesanan
7 FROM fact_penjualan fp
8 JOIN dim_waktu dw ON fp.id_waktu = dw.id_waktu
9 GROUP BY dw.tahun, dw.bulan
10 ORDER BY dw.tahun ASC, dw.bulan ASC;
```

Query melakukan proses JOIN antara tabel Fact_Penjualan dan Dim_Waktu menggunakan kolom id_waktu agar data transaksi dapat dikelompokkan berdasarkan tahun dan bulan. Fungsi SUM(fp.harga_barang) digunakan untuk menghitung total nilai penjualan setiap bulan. Fungsi COUNT(fp.id_fakta) digunakan untuk menghitung jumlah item yang terjual, sedangkan COUNT(DISTINCT fp.id_pesanan) digunakan untuk menghitung jumlah pesanan yang berbeda. Berikut adalah hasil query untuk KPI Tren Penjualan Bulanan dan Volume Transaksi:

tahun	bulan	total_penjualan	jumlah_item_terjual	jumlah_pesanan
2016	9	267.36	6	3
2016	10	49507.66	363	308
2016	12	10.90	1	1
2017	1	120312.87	955	789
2017	2	247303.02	1951	1733
2017	3	374344.30	3000	2641
2017	4	359927.23	2684	2391
2017	5	506071.14	4136	3660
2017	6	433038.60	3583	3217
2017	7	498031.48	4519	3969
2017	8	573971.68	4910	4293
2017	9	624401.69	4831	4243
2017	10	664219.43	5322	4568
2017	11	1010271.37	8665	7451
2017	12	743914.17	6308	5624
2018	1	950030.36	8208	7220
2018	2	844178.71	7672	6694
2018	3	983213.44	8217	7188
2018	4	996647.75	7975	6934
2018	5	996517.68	7925	6853
2018	6	865124.31	7078	6160
2018	7	895507.22	7092	6273
2018	8	854686.33	7248	6452
2018	9	145.00	1	1

Berdasarkan hasil query, total penjualan tertinggi terjadi pada November 2017, yaitu sebesar 1.010.271,37, dengan 8.665 item terjual dan 7.451 pesanan. Hasil tersebut menunjukkan bahwa November 2017 merupakan periode dengan aktivitas penjualan tertinggi. Informasi ini dapat digunakan untuk menentukan periode promosi dan mempersiapkan stok pada bulan dengan permintaan tinggi.

2. Lima Kategori Produk dengan Penjualan Tertinggi

```

1 SELECT
2     dp.kategori_produk,
3     COUNT(fp.id_fakta) AS jumlah_item_terjual,
4     COUNT(DISTINCT fp.id_pesanan) AS jumlah_pesanan,
5     ROUND(SUM(fp.harga_barang), 2) AS total_penjualan,
6     ROUND(AVG(fp.harga_barang), 2) AS rata_rata_harga_item
7 FROM fact_penjualan fp
8 JOIN dim_produk dp ON fp.id_produk = dp.id_produk
9 GROUP BY dp.kategori_produk
10 ORDER BY total_penjualan DESC
11 LIMIT 5;

```

Proses analisis dilakukan dengan menggabungkan tabel Fact_Penjualan dan Dim_Produk melalui kolom id_produk. Data kemudian dikelompokkan berdasarkan kategori_produk. Fungsi COUNT(fp.id_fakta) digunakan untuk menghitung jumlah item terjual, sedangkan COUNT(DISTINCT fp.id_pesanan) digunakan untuk mengetahui jumlah pesanan yang mengandung produk dari kategori tersebut. Selain itu, fungsi SUM(fp.harga_barang) digunakan untuk menghitung total penjualan dan fungsi AVG(fp.harga_barang) digunakan untuk menghitung rata-rata harga item. Hasil query diurutkan berdasarkan total penjualan tertinggi dan dibatasi menggunakan LIMIT 5, sehingga hanya lima kategori terbaik yang ditampilkan. Berikut adalah hasil query untuk KPI Lima Kategori Produk dengan Penjualan Tertinggi:

kategori_produk	jumlah_item_terjual	jumlah_pesanan	total_penjualan ▾ 1	rata_rata_harga_item
health_beauty	9670	8836	1258681.34	130.16
watches_gifts	5991	5624	1205005.68	201.14
bed_bath_table	11115	9417	1036988.68	93.30
sports_leisure	8641	7720	988048.97	114.34
computers_accessories	7827	6689	911954.32	116.51

Hasil query menunjukkan kategori health_beauty memiliki total penjualan tertinggi sebesar 1.258.681,34 dengan 9.670 item terjual. Selanjutnya, kategori watches_gifts memperoleh total penjualan sebesar 1.205.005,68. Kedua kategori tersebut dapat diprioritaskan dalam kegiatan promosi dan pengelolaan stok.

3. Kinerja Penjual Berdasarkan Total Penjualan

```

1 SELECT
2     dp.id_penjual,
3     dp.kota_penjual,
4     dp.provinsi_penjual,
5     SUM(fp.harga_barang) AS total_penjualan
6 FROM fact_penjualan fp
7 JOIN dim_penjual dp ON fp.id_penjual = dp.id_penjual
8 GROUP BY
9     dp.id_penjual,
10    dp.kota_penjual,
11    dp.provinsi_penjual
12 ORDER BY total_penjualan DESC
13 LIMIT 10;

```

Query menggabungkan tabel Fact_Penjualan dengan Dim_Penjual menggunakan kolom id_penjual. Setelah proses penggabungan, data dikelompokkan berdasarkan identitas penjual, kota, dan provinsi asal penjual. Fungsi SUM(fp.harga_barang) digunakan untuk menghitung seluruh nilai penjualan yang berhasil dihasilkan oleh masing-masing penjual. Hasil query kemudian diurutkan dari total penjualan tertinggi dan dibatasi menjadi sepuluh penjual menggunakan LIMIT 10.

id_penjual	kota_penjual	provinsi_penjual	total_penjualan ▾ 1
4869f7a5dfa277a7dca6462dcf3b52b2	guariba	SP	229472.63
53243585a1d6dc2643021fd1853d8905	lauro de freitas	BA	222776.05
4a3ca9315b744ce9f8e9374361493884	ibitinga	SP	200472.92
fa1c13f2614d7b5c4749cbc52fecda94	sumare	SP	194042.03
7c67e1448b00f6e969d365cea6b010ab	itaquaquecetuba	SP	187923.89
7e93a43ef30c4f03f38b393420bc753a	barueri	SP	176431.87
da8622b14eb17ae2831f4ac5b9dab84a	piracicaba	SP	160236.57
7a67c85e85bb2ce8582c35f2203ad736	sao paulo	SP	141745.53
1025f0e2d44d7041d6cf58b6550e0bfa	sao paulo	SP	138968.55
955fee9216a65b617aa5c0531780ce60	sao paulo	SP	135171.70

Hasil query menunjukkan penjual dengan total penjualan tertinggi berasal dari Kota Guariba, Provinsi SP, dengan nilai penjualan sebesar 229.472,63. Posisi kedua

ditempati oleh penjual dari Kota Lauro de Freitas, Provinsi BA, dengan total penjualan sebesar 222.776,05. Sementara itu, posisi ketiga ditempati oleh penjual dari Kota Ibitinga, Provinsi SP, dengan total penjualan sebesar 200.472,92. Informasi ini dapat digunakan untuk memberikan penghargaan atau dukungan promosi kepada penjual yang memiliki kontribusi besar.

4. Total Ongkos Kirim per Kuartal

```

1 SELECT
2     dw.tahun,
3     dw.kuartal,
4     SUM(fp.ongkos_kirim) AS total_ongkos_kirim,
5     SUM(fp.harga_barang) AS total_penjualan,
6     ROUND(SUM(fp.ongkos_kirim) / SUM(fp.harga_barang) * 100, 2) AS rasio_ongkir_persen
7 FROM fact_penjualan fp
8 JOIN dim_waktu dw ON fp.id_waktu = dw.id_waktu
9 GROUP BY dw.tahun, dw.kuartal
10 ORDER BY dw.tahun ASC, dw.kuartal ASC;

```

Query melakukan proses JOIN antara Fact_Penjualan dan Dim_Waktu, kemudian mengelompokkan data berdasarkan tahun dan kuartal. Fungsi SUM(fp.ongkos_kirim) digunakan untuk menghitung total ongkos kirim, sedangkan SUM(fp.harga_barang) digunakan untuk menghitung total penjualan. Query juga menghitung rasio ongkos kirim dengan membagi total ongkos kirim terhadap total penjualan, kemudian dikalikan 100 untuk menghasilkan nilai persentase. Rasio tersebut digunakan untuk melihat seberapa besar biaya pengiriman dibandingkan dengan nilai produk yang terjual pada setiap periode.

tahun	kuartal	total_ongkos_kirim	total_penjualan	rasio_ongkir_persen
2016	Q3	87.39	267.36	32.69
2016	Q4	7309.90	49518.56	14.76
2017	Q1	113557.51	741960.19	15.31
2017	Q2	202539.26	1299036.97	15.59
2017	Q3	277170.28	1696404.85	16.34
2017	Q4	393598.40	2418404.97	16.28
2018	Q1	471915.16	2777422.51	16.99
2018	Q2	473867.23	2858289.74	16.58
2018	Q3	311864.41	1750338.55	17.82

Berdasarkan hasil query, total ongkos kirim tertinggi terjadi pada kuartal kedua tahun 2018, yaitu sebesar 473.867,23, dengan total penjualan sebesar 2.858.289,74. Hasil ini menunjukkan bahwa peningkatan penjualan juga diikuti oleh peningkatan biaya pengiriman.

5. Total Penjualan Berdasarkan Provinsi Pelanggan

```
1 SELECT
2     dc.provinsi_pelanggan,
3     COUNT(DISTINCT fp.id_pesanan) AS jumlah_pesanan,
4     SUM(fp.harga_barang) AS total_penjualan,
5     COUNT(DISTINCT dc.id_pelanggan) AS jumlah_pelanggan
6 FROM fact_penjualan fp
7 JOIN dim_pelanggan dc ON fp.id_pelanggan = dc.id_pelanggan
8 GROUP BY dc.provinsi_pelanggan
9 ORDER BY total_penjualan DESC;
```

Query menggabungkan tabel Fact_Penjualan dengan Dim_Pelanggan melalui kolom id_pelanggan. Fungsi COUNT(DISTINCT fp.id_pesanan) digunakan untuk menghitung jumlah pesanan yang berbeda pada setiap provinsi. Fungsi SUM(fp.harga_barang) digunakan untuk menghitung total penjualan, sedangkan COUNT(DISTINCT dc.id_pelanggan) digunakan untuk menghitung jumlah identitas pelanggan yang berbeda. Data kemudian dikelompokkan berdasarkan provinsi pelanggan dan diurutkan dari total penjualan tertinggi.

provinsi_pelanggan	jumlah_pesanan	total_penjualan	jumlah_pelanggan
SP	41375	5202955.05	41375
RJ	12762	1824092.67	12762
MG	11544	1585308.03	11544
RS	5432	750304.02	5432
PR	4998	683083.76	4998
SC	3612	520553.34	3612
BA	3358	511349.99	3358
DF	2125	302603.94	2125
GO	2007	294591.95	2007
ES	2025	275037.31	2025
PE	1648	262788.03	1648
CE	1327	227254.71	1327
PA	970	178947.81	970
MT	903	156453.53	903
MA	740	119648.22	740
MS	709	116812.64	709
PB	532	115268.08	532
PI	493	86914.08	493
RN	482	83034.98	482
AL	411	80314.81	411
SE	345	58920.85	345
TO	279	49621.74	279
RO	247	46140.64	247
AM	147	22356.84	147
AC	81	15982.95	81
AP	68	13474.30	68
RR	46	7829.43	46

Hasil query menunjukkan provinsi SP menghasilkan total penjualan tertinggi sebesar 5.202.955,05 dari 41.375 pesanan. Posisi kedua ditempati Provinsi RJ dengan total penjualan sebesar 1.824.092,67. Hasil tersebut menunjukkan bahwa aktivitas penjualan Olist masih didominasi oleh pelanggan dari Provinsi SP.

BAB V

PENUTUP

5.1 Kesimpulan

Berdasarkan project yang telah dilakukan, data warehouse penjualan Olist berhasil dirancang menggunakan pendekatan star schema yang terdiri dari tabel Dim_Pelanggan, Dim_Penjual, Dim_Produk, Dim_Waktu, dan Fact_Penjualan. Struktur tersebut dapat mengintegrasikan data pelanggan, produk, penjual, waktu, dan transaksi ke dalam satu penyimpanan yang terstruktur sehingga mempermudah proses analisis.

Proses ETL berhasil diterapkan menggunakan Pentaho Data Integration dengan tahapan extract dari database db_oltpolist, transformasi data, serta load ke database dw_penjualanolist. Seluruh transformasi juga berhasil dijalankan secara berurutan melalui Job ETL tanpa mengalami kesalahan. Hasilnya, data warehouse berhasil memuat 99.441 data pelanggan, 3.095 data penjual, 32.951 data produk, 634 data waktu, dan 112.650 data fakta penjualan.

Berdasarkan analisis KPI, penjualan Olist mengalami peningkatan yang cukup tinggi, dengan aktivitas penjualan bulanan tertinggi terjadi pada November 2017. Kategori health_beauty menjadi kategori dengan total penjualan terbesar, sedangkan mayoritas penjual berkinerja tinggi dan pelanggan dengan kontribusi penjualan terbesar berasal dari Provinsi SP. Analisis ongkos kirim juga menunjukkan bahwa peningkatan penjualan diikuti oleh peningkatan biaya pengiriman.

Secara keseluruhan, data warehouse yang dibangun telah mampu menghasilkan informasi yang dapat digunakan untuk mengetahui tren penjualan, menentukan kategori produk unggulan, mengevaluasi kinerja penjual, memantau biaya pengiriman, serta mengetahui persebaran pasar. Informasi tersebut dapat membantu dalam mendukung proses pengambilan keputusan bisnis yang lebih tepat.

LAMPIRAN

Link Github:

https://github.com/Fizzrss/kelompok_2_project_uas_dw